

TA2 Actividades de modelos lineales

Índice

1. Introducción
2. Análisis discriminante cuadrático (QDA)
3. Análisis discriminante lineal (LDA)
4. Regresión logística binaria
5. Regresión logística
6. Regresión lineal

1 Introducción

☐ Los clasificadores generativos básicos expresan **posteriors** en función de **priors** y **densidades condicionales** de las clases. En relación con ello, indica la expresión incorrecta o escoge la última opción si las tres primeras son correctas:

1. $p(y = c \mid \mathbf{x}, \boldsymbol{\theta}) = \frac{p(\mathbf{x}, y = c \mid \boldsymbol{\theta})}{p(\mathbf{x} \mid \boldsymbol{\theta})}$
2. $p(y = c \mid \mathbf{x}, \boldsymbol{\theta}) = \frac{p(\mathbf{x}, y = c \mid \boldsymbol{\theta})}{\sum_{c'} p(\mathbf{x}, y = c' \mid \boldsymbol{\theta})}$
3. $p(y = c \mid \mathbf{x}, \boldsymbol{\theta}) = \frac{p(\mathbf{x} \mid y = c, \boldsymbol{\theta}) p(y = c, \boldsymbol{\theta})}{\sum_{c'} p(\mathbf{x} \mid y = c', \boldsymbol{\theta}) p(y = c', \boldsymbol{\theta})}$
4. Las tres afirmaciones anteriores son correctas.

Solución: La 4.

☐ A continuación se describen algunas ventajas bien conocidas de los clasificadores generativos usuales. En relación con las mismas, indica la afirmación incorrecta o escoge la última opción si las tres primeras son correctas:

1. Son fáciles de ajustar.
2. Pueden extenderse a nuevas clases sin necesidad de re-entrenar parámetros ya entrenados.
3. Facilitan el tratamiento de datos perdidos.
4. Las tres afirmaciones anteriores son correctas.

Solución: La 4.

☐ A continuación se describen algunas ventajas bien conocidas de los clasificadores discriminativos usuales. En relación con las mismas, indica la afirmación incorrecta o escoge la última opción si las tres primeras son correctas:

1. Suelen ser más precisos que los generativos, con posteriors mejor calibradas.
2. Facilitan el preproceso de características, típicamente mediante expansión polinómica.
3. Las redes usuales pueden verse como extensiones de modelos discriminativos básicos.
4. Las tres afirmaciones anteriores son correctas.

Solución: La 4.

2 Análisis discriminante cuadrático (QDA)

Problema. Sea un problema de clasificación en $C = 2$ clases equiprobables de datos representados mediante vectores de $D = 2$ reales. Se tiene un clasificador QDA con condicionales de las clases

$$\begin{aligned} p(\mathbf{x} \mid y = 1, \boldsymbol{\theta}) &= \mathcal{N}_D(\mathbf{x} \mid \boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1) & \boldsymbol{\mu}_1 &= (0, 0)^t & \boldsymbol{\Sigma}_1 &= \mathbf{I} \\ p(\mathbf{x} \mid y = 2, \boldsymbol{\theta}) &= \mathcal{N}_D(\mathbf{x} \mid \boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2) & \boldsymbol{\mu}_2 &= (2, 0)^t & \boldsymbol{\Sigma}_2 &= \text{diag}(1/5, 1/4) \end{aligned}$$

La regla de decisión MAP induce una frontera de separación elíptica de centro $(c_1, c_2)^t$ y semiejes a y b :

$$\frac{(x_1 - c_1)^2}{a^2} + \frac{(x_2 - c_2)^2}{b^2} = 1$$

Determina el centro y semiejes de la frontera.

Solución.

$$\log p(y = c \mid \mathbf{x}, \boldsymbol{\theta}) \propto \mathbf{x}^t \mathbf{A}_c \mathbf{x} + \boldsymbol{\beta}_c^t \mathbf{x} + \gamma_c$$

$$\mathbf{A}_c = -\frac{1}{2} \boldsymbol{\Sigma}_c^{-1} \quad \boldsymbol{\beta}_c = \boldsymbol{\Sigma}_c^{-1} \boldsymbol{\mu}_c \quad \gamma_c = -\frac{1}{2} \log |\boldsymbol{\Sigma}_c| - \frac{1}{2} \boldsymbol{\mu}_c^t \boldsymbol{\Sigma}_c^{-1} \boldsymbol{\mu}_c$$

$$\mathbf{A}_1 = -\frac{1}{2} \mathbf{I} \quad \boldsymbol{\beta}_1 = \mathbf{0} \quad \gamma_1 = 0$$

$$\mathbf{A}_2 = -\frac{1}{2} \text{diag}(5, 4) \quad \boldsymbol{\beta}_2 = (10, 0)^t \quad \gamma_2 = \frac{1}{2} \log 20 - 10 = -8.5$$

$$-\frac{1}{2} x_1^2 - \frac{1}{2} x_2^2 = -\frac{5}{2} x_1^2 - \frac{4}{2} x_2^2 + 10x_1 - 8.5$$

$$-x_1^2 - x_2^2 = -5x_1^2 - 4x_2^2 + 20x_1 - 17$$

$$4x_1^2 - 20x_1 + 3x_2^2 = -17$$

$$4(x_1^2 - 5x_1 + 25/4) - 25 + 3x_2^2 = -17$$

$$\frac{(x_1 - 5/2)^2}{1/4} + \frac{(x_2 - 0)^2}{1/3} = 8$$

$$\frac{(x_1 - 5/2)^2}{2} + \frac{(x_2 - 0)^2}{8/3} = 1$$

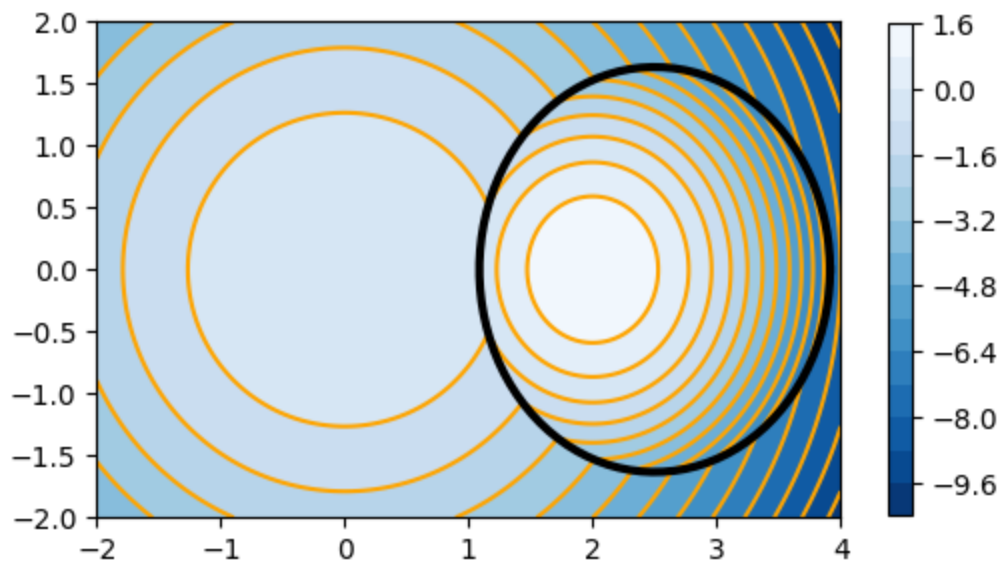
$$c_1 = 2.5 \quad c_2 = 0 \quad a = \sqrt{2} = 1.4142 \quad b = \sqrt{8/3} = 1.6330$$

```

In [ ]: import numpy as np; import matplotlib.pyplot as plt
m1 = np.zeros(2); S1 = np.eye(2); iS1 = np.linalg.inv(S1)
m2 = np.array([2, 0]); S2 = np.diag([.2, .25]); iS2 = np.linalg.inv(S2)
A1, b1, c1 = -0.5 * iS1, iS1 @ m1, -0.5 * np.log(np.linalg.det(S1)) - 0.5 * m1.T @ iS1 @ m1
A2, b2, c2 = -0.5 * iS2, iS2 @ m2, -0.5 * np.log(np.linalg.det(S2)) - 0.5 * m2.T @ iS2 @ m2
x1, x2 = np.meshgrid(np.linspace(-2, 4, 200), np.linspace(-2, 2, 200))
x = np.c_[np.ravel(x1), np.ravel(x2)]
p1 = lambda x: x.T @ A1 @ x + b1 @ x + c1
p2 = lambda x: x.T @ A2 @ x + b2 @ x + c2
maxp = lambda x: max(p1(x), p2(x))
maxpx = np.apply_along_axis(maxp, 1, x)
amaxp = lambda x: np.argmax([p1(x), p2(x)])
amaxpx = np.apply_along_axis(amaxp, 1, x);

In [ ]: fig, ax = plt.subplots(1, 1, figsize=(6, 4)); ax.set(aspect='equal')
ax.contour(x1, x2, maxpx.reshape(x1.shape), 16, colors='orange', linestyle='solid')
cp = ax.contourf(x1, x2, maxpx.reshape(x1.shape), 16, cmap='Blues_r')
plt.colorbar(cp, ax=ax, shrink=0.8)
t = np.linspace(0, 2.0 * np.pi, 100)
ellipse = np.array([1.4142 * np.cos(t) + 2.5, 1.633 * np.sin(t)]).T
ax.plot(ellipse[:, 0], ellipse[:, 1], color='black', lw=3);
#ax.contour(x1, x2, amaxpx.reshape(x1.shape), colors='white', linestyle='solid', linewidths=1);

```



Problema. Sea un problema de clasificación en $C = 2$ clases equiprobables de datos representados mediante vectores de $D = 2$ reales. Se tiene un clasificador QDA con condicionales de las clases

$$\begin{aligned} p(\mathbf{x} \mid y = 1, \boldsymbol{\theta}) &= \mathcal{N}_D(\mathbf{x} \mid \boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1) & \boldsymbol{\mu}_1 &= (0, 0)^t & \boldsymbol{\Sigma}_1 &= \text{diag}(1, 1/9) \\ p(\mathbf{x} \mid y = 2, \boldsymbol{\theta}) &= \mathcal{N}_D(\mathbf{x} \mid \boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2) & \boldsymbol{\mu}_2 &= (2, 0)^t & \boldsymbol{\Sigma}_2 &= \text{diag}(1/9, 1) \end{aligned}$$

La regla de decisión MAP induce una frontera de separación hiperbólica de centro $(c_1, c_2)^t$ y semiejes a y b :

$$\frac{(x_1 - c_1)^2}{a^2} - \frac{(x_2 - c_2)^2}{b^2} = 1$$

Determina el centro y semiejes de la frontera.

Solución.

$$\log p(y = c \mid \mathbf{x}, \boldsymbol{\theta}) \propto \mathbf{x}^t \mathbf{A}_c \mathbf{x} + \boldsymbol{\beta}_c^t \mathbf{x} + \gamma_c$$

$$\mathbf{A}_c = -\frac{1}{2} \boldsymbol{\Sigma}_c^{-1} \quad \boldsymbol{\beta}_c = \boldsymbol{\Sigma}_c^{-1} \boldsymbol{\mu}_c \quad \gamma_c = -\frac{1}{2} \log |\boldsymbol{\Sigma}_c| - \frac{1}{2} \boldsymbol{\mu}_c^t \boldsymbol{\Sigma}_c^{-1} \boldsymbol{\mu}_c$$

$$\mathbf{A}_1 = -\frac{1}{2} \text{diag}(1, 9) \quad \boldsymbol{\beta}_1 = \mathbf{0} \quad \gamma_1 = \frac{1}{2} \log 9 = 1.0986$$

$$\mathbf{A}_2 = -\frac{1}{2} \text{diag}(9, 1) \quad \boldsymbol{\beta}_2 = (18, 0)^t \quad \gamma_2 = \frac{1}{2} \log 9 - 18 = -16.9014$$

$$-\frac{1}{2} x_1^2 - \frac{9}{2} x_2^2 + 1.0986 = -\frac{9}{2} x_1^2 - \frac{1}{2} x_2^2 + 18x_1 - 16.9014$$

$$-x_1^2 - 9x_2^2 + 2.1972 = -9x_1^2 - x_2^2 + 36x_1 - 33.8028$$

$$8x_1^2 - 36x_1 - 8x_2^2 = -36$$

$$x_1^2 - 9/2 x_1 - x_2^2 = -9/2$$

$$(x_1 - 9/4)^2 - 81/16 - x_2^2 = -9/2$$

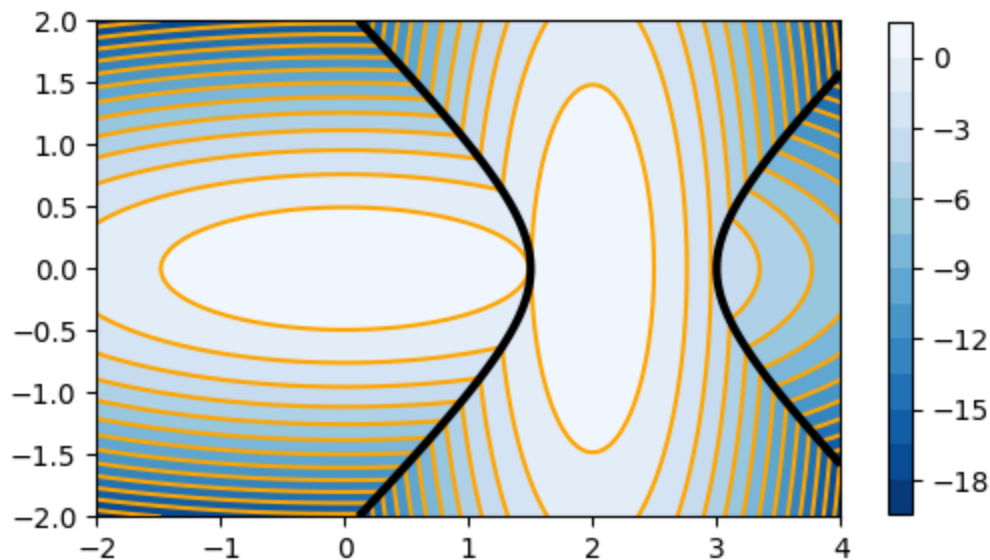
$$(x_1 - 9/4)^2 - x_2^2 = 9/16$$

$$\frac{(x_1 - 9/4)^2}{9/16} - \frac{(x_2 - 0)^2}{9/16} = 1$$

$$c_1 = 9/4 \quad c_2 = 0 \quad a = b = \sqrt{9/16} = 3/4$$

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt
m1 = np.zeros(2); S1 = np.diag([1, 1/9]); iS1 = np.linalg.inv(S1)
m2 = np.array([2, 0]); S2 = np.diag([1/9, 1]); iS2 = np.linalg.inv(S2)
A1, b1, c1 = -0.5 * iS1, iS1 @ m1, -0.5 * np.log(np.linalg.det(S1)) - 0.5 * m1.T @ iS1 @ m1
A2, b2, c2 = -0.5 * iS2, iS2 @ m2, -0.5 * np.log(np.linalg.det(S2)) - 0.5 * m2.T @ iS2 @ m2
x1, x2 = np.meshgrid(np.linspace(-2, 4, 200), np.linspace(-2, 2, 200))
x = np.c_[np.ravel(x1), np.ravel(x2)]
p1 = lambda x: x.T @ A1 @ x + b1 @ x + c1
p2 = lambda x: x.T @ A2 @ x + b2 @ x + c2
maxp = lambda x: max(p1(x), p2(x))
maxpx = np.apply_along_axis(maxp, 1, x)
amaxp = lambda x: np.argmax([p1(x), p2(x)])
amaxpx = np.apply_along_axis(amaxp, 1, x);
```

```
In [ ]: fig, ax = plt.subplots(1, 1, figsize=(6, 4)); ax.set(aspect='equal')
ax.contour(x1, x2, maxpx.reshape(x1.shape), 16, colors='orange', linestyle='solid')
cp = ax.contourf(x1, x2, maxpx.reshape(x1.shape), 16, cmap='Blues_r')
plt.colorbar(cp, ax=ax, shrink=0.8)
t = np.linspace(-np.pi/2.8, np.pi/2.8, 100)
hiperbola1 = np.array([3/4 / np.cos(t) + 9/4, 3/4 * np.tan(t)]).T
ax.plot(hiperbola1[:, 0], hiperbola1[:, 1], color='black', lw=3);
t = np.linspace(-np.pi/2.6, np.pi/2.6, 100)
hiperbola2 = np.array([-3/4 / np.cos(t) + 9/4, 3/4 * np.tan(t)]).T
ax.plot(hiperbola2[:, 0], hiperbola2[:, 1], color='black', lw=3);
#ax.contour(x1, x2, amaxpx.reshape(x1.shape), colors='white', linestyle='solid', linewidths=1);
```



3 Análisis discriminante lineal (LDA)

☐ Indica la afirmación incorrecta o escoge la última opción si las tres primeras son correctas:

1. QDA asume que las densidades condicionales de las clases son Gaussianas independientes, lo que resulta en discriminantes (log-posteriors) cuadráticas.
2. LDA asume que las densidades condicionales de las clases son Gaussianas de matriz de covarianzas común, lo que resulta en discriminantes (log-posteriors) lineales.
3. QDA suele ajustarse con las medias empíricas de las clases y la matriz de covarianzas empírica global.
4. Las tres afirmaciones anteriores son correctas.

Solución: La 3. QDA suele ajustarse con las medias y matrices de covarianzas empíricas de las clases. LDA suele ajustarse con las medias empíricas de las clases y la matriz de covarianzas empírica global.

☐ QDA, LDA y Naive Bayes Gaussiano (NBG) son tres modelos generativos en los que se asume que las densidades condicionales de las clases son Gaussianas de medias independientes. La principal diferencia entre los mismos reside en las asunciones adicionales que se realizan sobre las matrices de covarianzas de las Gaussianas. En relación con estas asunciones adicionales, indica la respuesta incorrecta o la última opción si las tres primeras son correctas.

1. QDA no realiza ninguna asunción adicional; esto es, cada Gaussiana viene gobernada por una matriz de covarianzas independiente.
2. LDA asume que todas las Gaussianas comparten una misma matriz de covarianzas común.
3. Como su nombre indica, NGB se basa en la asunción de Bayes, esto es, que las características son condicionalmente independientes dada la clase. En el caso Gaussiano, esto supone que NBG puede verse como QDA con matrices de covarianzas diagonales.
4. Las tres opciones anteriores son correctas.

Solución: La 4.

☐ Con respecto al clasificador por el centroide más próximo, indica la opción correcta:

1. Asume distribuciones condicionales Gaussianas, con una media y una matriz de covarianzas para cada clase. La clasificación se realiza a la clase con la menor distancia Euclídea entre el punto $\mathbf{x}$ a clasificar y la media μ_c .
2. Asume distribuciones condicionales Gaussianas, con una media y una matriz de covarianzas para cada clase y priors idénticas. La clasificación se realiza a la clase con la menor distancia de Mahalanobis entre el punto $\mathbf{x}$ a clasificar y la media μ_c .
3. Asume distribuciones condicionales Gaussianas, con una media y una matriz de covarianzas para cada clase. La clasificación se realiza a la clase con la menor distancia de Mahalanobis entre el punto $\mathbf{x}$ a clasificar y la media μ_c .
4. Asume distribuciones condicionales Gaussianas, con una media y una matriz de covarianzas para cada clase y priors idénticas. La clasificación se realiza a la clase con la menor distancia Euclídea entre el punto $\mathbf{x}$ a clasificar y la media μ_c .

Solución: La 2.

□ Sea un problema de clasificación de objetos representados mediante vectores de $D = 2$ variables reales en $C = 3$ clases de probabilidades a priori idénticas. Se tiene un clasificador Gaussiano de densidades condicionales con matrices de covarianzas común:

$$p(\mathbf{x} \mid y = c, \boldsymbol{\theta}) = \mathcal{N}_D(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}) \quad \text{donde} \quad \boldsymbol{\theta} = (\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_C, \text{vec}(\boldsymbol{\Sigma}))^t$$

con

$$\boldsymbol{\mu}_1 = (0, 0)^t, \quad \boldsymbol{\mu}_2 = (0, 4)^t, \quad \boldsymbol{\mu}_3 = (4, 4)^t \quad \text{y} \quad \boldsymbol{\Sigma} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

La región de decisión de la clase 2 es:

1. $\mathcal{R}_2 = \{\mathbf{x} : x_1 > 2, x_2 > 2\}$
2. $\mathcal{R}_2 = \{\mathbf{x} : x_1 > 2, x_2 < 2\}$
3. $\mathcal{R}_2 = \{\mathbf{x} : x_1 < 2, x_2 > 2\}$
4. $\mathcal{R}_2 = \{\mathbf{x} : x_1 < 2, x_2 < 2\}$

Solución: La 3. Discriminantes lineales simplificadas (ignorando términos aditivos independientes de c):

$$\log p(y = c \mid \mathbf{x}, \boldsymbol{\theta}) \propto g_c(\mathbf{x}) = \boldsymbol{\beta}_c^t \mathbf{x} + \gamma_c \quad \text{con} \quad \boldsymbol{\beta}_c = \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_c = \boldsymbol{\mu}_c \quad \text{y} \quad \gamma_c = -\frac{1}{2} \boldsymbol{\mu}_c^t \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_c = -\frac{1}{2} \boldsymbol{\mu}_c^t \boldsymbol{\mu}_c$$

$$g_1(\mathbf{x}) = \boldsymbol{\mu}_1^t \mathbf{x} - \frac{1}{2} \boldsymbol{\mu}_1^t \boldsymbol{\mu}_1 = 0$$

$$g_2(\mathbf{x}) = \boldsymbol{\mu}_2^t \mathbf{x} - \frac{1}{2} \boldsymbol{\mu}_2^t \boldsymbol{\mu}_2 = 4x_2 - 8$$

$$g_3(\mathbf{x}) = \boldsymbol{\mu}_3^t \mathbf{x} - \frac{1}{2} \boldsymbol{\mu}_3^t \boldsymbol{\mu}_3 = 4x_1 + 4x_2 - 16$$

$$\begin{aligned} \mathcal{R}_2 &= \{\mathbf{x} : g_2(\mathbf{x}) > g_1(\mathbf{x}), g_2(\mathbf{x}) > g_3(\mathbf{x})\} \\ &= \{\mathbf{x} : 4x_2 - 8 > 0, 4x_2 - 8 > 4x_1 + 4x_2 - 16\} \\ &= \{\mathbf{x} : x_1 < 2, x_2 > 2\} \end{aligned}$$

4 Regresión logística binaria

Problema. Sea un problema de clasificación en dos clases, $y \in \{0, 1\}$, y sea $p(y | \mathbf{x}; \boldsymbol{\theta}) = \text{Ber}(y | \sigma(\mathbf{w}^t \mathbf{x} + b))$ un modelo de regresión logística binaria con $\mathbf{w} = (3, 3)^t$ y $b = 0$. Determina la probabilidad de que $\mathbf{x} = (0.5, 0.5)^t$ pertenezca a la clase 1 según el modelo dado.

Solución:

$$p(y = 1 | \mathbf{x}; \boldsymbol{\theta}) = \sigma(\mathbf{w}^t \mathbf{x} + b) = \sigma((3, 3)(0.5, 0.5)^t + 0) = \frac{1}{1 + e^{-3}} = 0.9526$$

Problema. Sea un problema de clasificación en dos clases, $y \in \{0, 1\}$, y sea $p(y | \mathbf{x}; \boldsymbol{\theta}) = \text{Ber}(y | \sigma(\mathbf{w}^t \mathbf{x} + b))$ un modelo de regresión logística binaria con $\mathbf{w} = (3, 3)^t$ y $b = 0$. Determina la logit, $f(\mathbf{x}; \boldsymbol{\theta})$, así como la frontera y regiones de decisión que induce (con pérdida 0-1).

Solución:

Logit: $f(\mathbf{x}; \boldsymbol{\theta}) = b + \mathbf{w}^t \mathbf{x} = 3x_1 + 3x_2$

Frontera: $3x_1 + 3x_2 = 0 \rightarrow x_2 = -x_1$

Regiones de decisión:

$$\begin{aligned}\mathcal{R}_1 &= \{\mathbf{x} : p(y = 1 | \mathbf{x}; \boldsymbol{\theta}) > p(y = 0 | \mathbf{x}; \boldsymbol{\theta})\} \\ &= \left\{ \mathbf{x} : \frac{p(y = 1 | \mathbf{x}; \boldsymbol{\theta})}{p(y = 0 | \mathbf{x}; \boldsymbol{\theta})} > 1 \right\} \\ &= \{\mathbf{x} : f(\mathbf{x}; \boldsymbol{\theta}) > 0\} \\ &= \{\mathbf{x} : 3x_1 + 3x_2 > 0\} \\ &= \{\mathbf{x} : x_2 > -x_1\} \\ \mathcal{R}_0 &= \{\mathbf{x} : x_2 < -x_1\}\end{aligned}$$

□ Sea $\mathcal{D} = \{(\mathbf{x}_n, y_n) : \mathbf{x}_n \in \mathbb{R}^D, y_n \in \{0, 1\}\}$, un conjunto de muestras no linealmente separables. Se quiere aprender un modelo de clasificación $p(y | \mathbf{x}; \boldsymbol{\theta})$ que las clasifique con pérdida 01 nula. Indica la respuesta correcta:

1. Podemos usar regresión logística binaria convencional, esto es, con $p(y = 1 | \mathbf{x}) = \text{Ber}(y | \sigma(\mathbf{w}^t \mathbf{x} + b))$.
2. No podemos usar regresión logística binaria convencional, pero sí regresión logística binaria con una función de preproceso, $\phi(\mathbf{x})$, que linealice las muestras.
3. No podemos usar regresión logística binaria, convencional o no; necesitamos un modelo de clasificación multi-clase.
4. No podemos usar regresión logística binaria, convencional o no, pues es un modelo lineal y se necesita uno no lineal.

Solución: La 2; la 4 es falsa porque puede haber una función de preproceso que linealice las muestras.

Problema. Sea $p(y | \mathbf{x}; \mathbf{w}) = \text{Ber}(y | \sigma(\mathbf{w}^t \mathbf{x}))$, $y \in \{0, 1\}$, un modelo de regresión logística binaria en notación compacta (homogénea). Sea el conjunto de datos de entrenamiento $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}$, donde $\mathbf{x}_n \in \mathbb{R}^D$ y $y_n \in \{0, 1\}$. La entropía cruzada binaria de $\mathbf{w}$ es:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \ell(y_n, \hat{y}_n) \quad \text{con} \quad \ell(y_n, \hat{y}_n) = -y_n \log \hat{y}_n - (1 - y_n) \log(1 - \hat{y}_n)$$

Demuestra que el gradiente (transpuesto) de la entropía cruzada binaria es:

$$\frac{\partial \mathcal{L}(\mathbf{w})}{\partial \mathbf{w}} = \frac{1}{N} \sum_{n=1}^N (\hat{y}_n - y_n) \mathbf{x}_n^t \quad \text{donde} \quad \hat{y}_n = \mu_n = \sigma(a_n) \quad \text{con logodds} \quad a_n = \mathbf{w}^t \mathbf{x}_n$$

Solución. Sabemos que la derivada de la entropía cruzada binaria (con un dato) respecto de la logodds es la predicción menos la referencia. Así pues:

$$\frac{\partial \mathcal{L}(\mathbf{w})}{\partial \mathbf{w}} = \frac{1}{N} \sum_n \frac{\partial \ell(y_n, \hat{y}_n)}{\partial a_n} \frac{\partial a_n}{\partial \mathbf{w}} = \frac{1}{N} \sum_n (\hat{y}_n - y_n) \mathbf{x}_n^t = \frac{1}{N} (\boldsymbol{\sigma}(\mathbf{X}\mathbf{w}) - \mathbf{y})^t \mathbf{X} \quad \text{con} \quad \mathbf{y} = (y_1, \dots, y_N)^t$$

Problema. Sea $p(y | \mathbf{x}; \mathbf{w}) = \text{Ber}(y | \sigma(\mathbf{w}^t \mathbf{x}))$, $y \in \{0, 1\}$, un modelo de regresión logística binaria en notación compacta (homogénea). Sea el conjunto de datos de entrenamiento $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}$, donde $\mathbf{x}_n \in \mathbb{R}^D$ y $y_n \in \{0, 1\}$. Sabemos que el gradiente (transpuesto) de la entropía cruzada binaria es:

$$\frac{\partial \mathcal{L}(\mathbf{w})}{\partial \mathbf{w}} = \frac{1}{N} \sum_{n=1}^N (\hat{y}_n - y_n) \mathbf{x}_n^t \quad \text{donde} \quad \hat{y}_n = \mu_n = \sigma(a_n) \quad \text{con logodds} \quad a_n = \mathbf{w}^t \mathbf{x}_n$$

Supón que estamos aplicando SGD con minibatch de talla 1 para minimizar $\mathcal{L}(\mathbf{w})$. Más concretamente, nos hallamos en la iteración i con $\mathbf{w}_i = (-1, 0, -1)^t$, $\eta_i = 0.1$ y el minibatch actual únicamente incluye la muestra $\mathbf{x}_n = (1, 1, 1)^t$, de clase $y_n = 1$. Determina $\mathbf{w}_{i+1}$.

Solución:

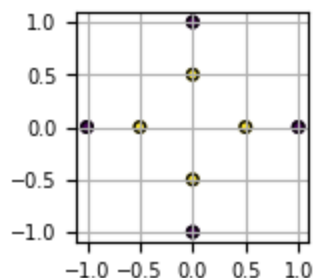
$$p(y_n = 1 | \mathbf{x}_n; \mathbf{w}_i) = \hat{y}_n = \mu_n = \sigma(\mathbf{w}_i^t \mathbf{x}_n) = \sigma(-2) = \frac{1}{1 + e^2} = 0.12$$

$$\begin{aligned} \mathbf{w}_{i+1} &= \mathbf{w}_i - \eta_i (\hat{y}_n - y_n) \mathbf{x}_n \\ &= (-1, 0, -1)^t + 0.1 (1 - 0.12)(1, 1, 1)^t \\ &= (-1, 0, -1)^t + 0.09 (1, 1, 1)^t \\ &= (-0.91, 0.09, -0.91)^t \end{aligned}$$

Problema. Sean los $N = 8$ datos de $D = 2$ dimensiones y $C = 2$ clases (0 y 1):

n	1	2	3	4	5	6	7	8
x_{n1}	1	0.5	0	0	-1	-0.5	0	0
x_{n2}	0	0	1	0.5	0	0	-1	-0.5
y_n	0	1	0	1	0	1	0	1

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt
X = np.array([[1, 0], [0.5, 0], [0, 1], [0, .5], [-1, 0], [-.5, 0], [0, -1], [0, -.5]])
y = np.array([0, 1, 0, 1, 0, 1, 0, 1])
_, ax = plt.subplots(figsize=(1.5, 1.5)); ax.grid()
ax.set_xticks(np.unique(X[:, 0])); ax.set_yticks(np.unique(X[:, 1])); ax.tick_params(axis='both', labels=7)
ax.scatter(*X.T, c=y, s=16); ax.scatter(*X.T, facecolors='none', edgecolors='black', s=16);
```



En vista de que los datos no son linealmente separables, se ha adoptado la siguiente función de preproceso:

$$\phi(x_1, x_2) = (1, x_1^2, x_2^2)$$

Con base en esta función, se ha considerado el siguiente modelo de regresión logística binaria:

$$p(y \mid \mathbf{x}, \mathbf{w}) = \text{Ber}(y \mid \mu)$$

con $\mu = \sigma(a)$ y log-odds lineal con la entrada preprocesada, $a = \mathbf{w}^t \phi(\mathbf{x})$.

Se pide:

1. Clasifica los datos por mínima probabilidad de error con $\mathbf{w} = (0.1, -0.3, 0)^t$.
2. Con el fin de minimizar la neg-log-verosimilitud de $\mathbf{w}$ respecto a los datos, se ha aplicado descenso por gradiente estocástico con $\mathbf{w}_0 = \mathbf{0}$ y $\eta_i = \eta = 0.75$, procesando uno a uno los datos en el orden dado, hasta obtener $\mathbf{w}_7 = (-0.1331, -0.5026, -0.6453)^t$ tras la séptima iteración. Calcula $\mathbf{w}_8$ sabiendo que el gradiente de la neg-log-verosimilitud para un único dato, $\mathbf{x}_n$, es $\mathbf{g}_n = (\mu_n - y_n)\phi(\mathbf{x}_n)$.
3. Halla la frontera entre las clases, en el espacio de representación original (2d), con $\mathbf{w}_8$.

Solución.

1. Clasificación de los datos por mínima probabilidad de error con $\mathbf{w} = (0.1, -0.3, 0)^t$:

$$y(\mathbf{x}) = \mathbb{I}(p(y = 1 | \mathbf{x}) > p(y = 0 | \mathbf{x})) = \mathbb{I}\left(\log \frac{p(y = 1 | \mathbf{x})}{p(y = 0 | \mathbf{x})} > 0\right) = \mathbb{I}(a > 0)$$

n	$\phi(\mathbf{x})^t$	a	$y(\mathbf{x}_n)$
1	(1, 1, 0)	$0.1 - 0.3 = -0.2$	0
2	(1, 0.25, 0)	$0.1 - 0.3 \cdot 0.25 = 0.025$	1
3	(1, 0, 1)	0.1	1
4	(1, 0, 0.25)	0.1	1
5	(1, 1, 0)	$0.1 - 0.3 = -0.2$	0
6	(1, 0.25, 0)	$0.1 - 0.3 \cdot 0.25 = 0.025$	1
7	(1, 0, 1)	0.1	1
8	(1, 0, 0.25)	0.1	1

2. Cálculo de $\mathbf{w}_8$:

$$\begin{aligned} a_8 &= \mathbf{w}_7^t \phi(\mathbf{x}_8) = (-0.1331, -0.5026, -0.6453)(1, 0, 0.25)^t \\ &= -0.1331 - 0.1613 = -0.2944 \end{aligned}$$

$$\mu_8 = \sigma(-0.2944) = \frac{1}{1 + e^{0.2944}} = 0.4269$$

$$\mathbf{g}_8 = (\mu_8 - y_8)\phi(\mathbf{x}_8) = (0.4269 - 1)(1, 0, 0.25)^t = (-0.5731, 0, -0.1433)^t$$

$$\begin{aligned} \mathbf{w}_8 &= \mathbf{w}_7 - \eta \mathbf{g}_8 = (-0.1331, -0.5026, -0.6453)^t - 0.75(-0.5731, 0, -0.1433)^t \\ &= (0.2967, -0.5026, -0.5378)^t \end{aligned}$$

3. La frontera es, aproximadamente, una circunferencia centrada en el origen de radio 0.75:

$$\begin{aligned} \mathbf{w}_8^t \phi(\mathbf{x}) &= 0 \\ (0.2967, -0.5026, -0.5378)(1, x_1^2, x_2^2) &= 0 \\ 0.5026 \cdot x_1^2 + 0.5378 \cdot x_2^2 &= 0.2967 \\ \frac{x_1^2}{r_1^2} + \frac{x_2^2}{r_2^2} &= 1 \quad \text{con} \quad r_1 = \sqrt{\frac{0.2967}{0.5026}} = 0.7683 \quad \text{y} \quad r_2 = \sqrt{\frac{0.2967}{0.5378}} = 0.7428 \end{aligned}$$

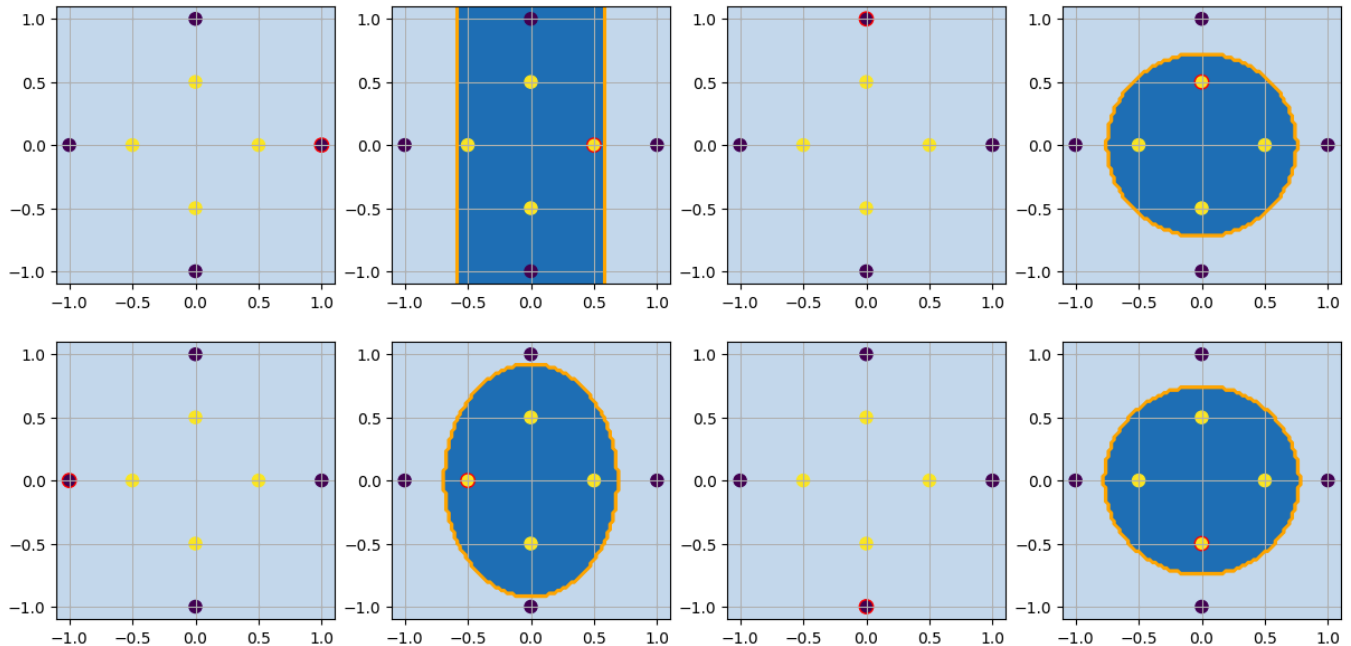
Preparación del problema y solución a máquina.

```
In [ ]: # phi = lambda x: np.array([1.0, x[0], x[1], x[0]**2, x[1]**2])
phi = lambda x: np.array([1.0, x[0]**2, x[1]**2])
Xp = np.apply_along_axis(phi, 1, X); D = Xp.shape[1]
```

```
In [ ]: N = 8; w, eta = np.zeros((9, D)), 0.75
for i in range(8):
    n = i % N
    mun = 1.0 / (1.0 + np.exp(- w[i, :] @ Xp[n, :]))
    grad = mun - y[n]
    w[i+1, :] = w[i, :] - eta * (mun - y[n]) * Xp[n, :]
    print(i+1, w[i+1])
```

```
1 [-0.375 -0.375  0.        ]
2 [ 0.08631589 -0.25967103  0.          ]
3 [-0.3048583  -0.25967103 -0.39117419]
4 [ 0.14463517 -0.25967103 -0.27880082]
5 [-0.20881936 -0.61312556 -0.27880082]
6 [ 0.23334229 -0.50258515 -0.27880082]
7 [-0.1331357  -0.50258515 -0.64527881]
8 [ 0.2966792  -0.50258515 -0.53782509]
```

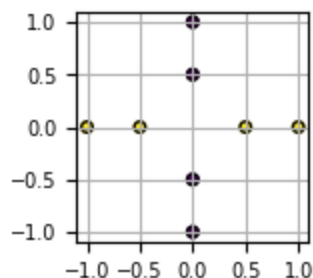
```
In [ ]: x_min, x_max = ax.get_xlim(); y_min, y_max = ax.get_ylim()
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100), np.linspace(y_min, y_max, 100))
XX = np.c_[np.ravel(xx), np.ravel(yy)]
fig, axes = plt.subplots(2, 4, figsize=(12, 6), constrained_layout=True)
for i in range(8):
    ax = axes.flat[i]; ax.axis('off'); ax = fig.add_subplot(2, 4, i + 1)
    ax.grid(); ax.set(aspect='equal')
    z = lambda x: w[i + 1, :] @ phi(x)
    zz = np.heaviside(np.apply_along_axis(z, 1, XX), 0.0)
    ax.contour(xx, yy, zz.reshape(xx.shape), 1, colors='orange', linestyle='solid')
    cp = ax.contourf(xx, yy, zz.reshape(xx.shape), 1, cmap='Blues')
    ax.scatter(*X.T, c=y, s=64); n = i % N
    ax.scatter(X[n, 0], X[n, 1], facecolors='none', edgecolors='red', s=80)
```



Problema. Sean los $N = 8$ datos de $D = 2$ dimensiones y $C = 2$ clases (0 y 1):

n	1	2	3	4	5	6	7	8
x_{n1}	-1	-0.5	0	0	0	0	0.5	1
x_{n2}	0	0	-1	-0.5	0.5	1	0	0
y_n	1	1	0	0	0	0	1	1

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt
X = np.array([[ -1, 0], [ -0.5, 0], [ 0, -1], [ 0, -0.5], [ 0, 0.5], [ 0, 1], [ 0.5, 0], [ 1, 0]])
y = np.array([1, 1, 0, 0, 0, 0, 1, 1]); _, ax = plt.subplots(figsize=(1.5, 1.5)); ax.grid()
ax.set_xticks(np.unique(X[:, 0])); ax.set_yticks(np.unique(X[:, 1])); ax.tick_params(axis='both', labelsz=7)
ax.scatter(*X.T, c=y, s=16); ax.scatter(*X.T, facecolors='none', edgecolors='black', s=16);
```



En vista de que los datos no son linealmente separables, se ha adoptado la siguiente función de preproceso:

$$\phi(x_1, x_2) = (1, x_1^2, x_2^2)$$

Con base en esta función, se ha considerado el siguiente modelo de regresión logística binaria:

$$p(y \mid \mathbf{x}, \mathbf{w}) = \text{Ber}(y \mid \mu)$$

con $\mu = \sigma(a)$ y log-odds lineal con la entrada preprocesada, $a = \mathbf{w}^t \phi(\mathbf{x})$.

Se pide:

1. Clasifica los datos por mínima probabilidad de error con $\mathbf{w} = (-0.2, 0.4, -0.6)^t$.
2. Con el fin de minimizar la neg-log-verosimilitud de $\mathbf{w}$ respecto a los datos, se ha aplicado descenso por gradiente estocástico con $\mathbf{w}_0 = \mathbf{0}$ y $\eta_i = \eta = 0.75$, procesando uno a uno los datos en el orden dado, hasta obtener $\mathbf{w}_7 = (-0.2157, 0.5676, -0.8427)^t$ tras la séptima iteración. Calcula $\mathbf{w}_8$ sabiendo que el gradiente de la neg-log-verosimilitud para un único dato, $\mathbf{x}_n$, es $\mathbf{g}_n = (\mu_n - y_n)\phi(\mathbf{x}_n)$.
3. Halla la frontera entre las clases, en el espacio de representación original (2d), con $\mathbf{w}_8$.

Solución.

1. Clasificación de los datos por mínima probabilidad de error con $\mathbf{w} = (-0.2, 0.4, -0.6)^t$.

$$y(\mathbf{x}) = \mathbb{I}(p(y = 1 | \mathbf{x}) > p(y = 0 | \mathbf{x})) = \mathbb{I}\left(\log \frac{p(y = 1 | \mathbf{x})}{p(y = 0 | \mathbf{x})} > 0\right) = \mathbb{I}(a > 0)$$

n	$\phi(\mathbf{x})^t$	a	$y(\mathbf{x}_n)$
1	(1, 1, 0)	$-0.2 + 0.4 = 0.2$	1
2	(1, 0.25, 0)	$-0.2 + 0.4 \cdot 0.25 = -0.1$	0
3	(1, 0, 1)	$-0.2 - 0.6 = -0.8$	0
4	(1, 0, 0.25)	$-0.2 - 0.6 \cdot 0.25 = -0.35$	0
5	(1, 0, 0.25)	-0.35	0
6	(1, 0, 1)	-0.8	0
7	(1, 0.25, 0)	-0.1	0
8	(1, 1, 0)	0.2	1

2. Cálculo de $\mathbf{w}_8$:

$$\begin{aligned} a_8 &= \mathbf{w}_7^t \phi(\mathbf{x}_8) = (-0.2157, 0.5676, -0.8427)(1, 1, 0)^t \\ &= -0.2157 + 0.5676 = 0.3519 \end{aligned}$$

$$\mu_8 = \sigma(0.3519) = \frac{1}{1 + e^{-0.3519}} = 0.5871$$

$$\mathbf{g}_8 = (\mu_8 - y_8)\phi(\mathbf{x}_8) = (0.5871 - 1)(1, 1, 0)^t = (-0.4129, -0.4129, 0)^t$$

$$\mathbf{w}_8 = \mathbf{w}_7 - \eta \mathbf{g}_8 = (-0.2157, 0.5676, -0.8427)^t - 0.75(-0.4129, -0.4129, 0)^t = (0.0940, 0.8773, -0.8427)^t$$

3. La frontera es una hipérbola vertical centrada en el origen, vértices $(0, \pm a)$ y focos $(0, \pm c)$, $c = \sqrt{a^2 + b^2} = 0.47$

$$\begin{aligned} \mathbf{w}_8^t \phi(\mathbf{x}) &= 0 \\ (0.0940, 0.8773, -0.8427)(1, x_1^2, x_2^2)^t &= 0 \\ -0.8773 x_1^2 + 0.8427 x_2^2 &= 0.0940 \\ \frac{x_2^2}{a^2} - \frac{x_1^2}{b^2} &= 1 \quad \text{con} \quad a = \sqrt{\frac{0.0940}{0.8427}} = 0.3340 \quad \text{y} \quad b = \sqrt{\frac{0.0940}{0.8773}} = 0.3273 \end{aligned}$$

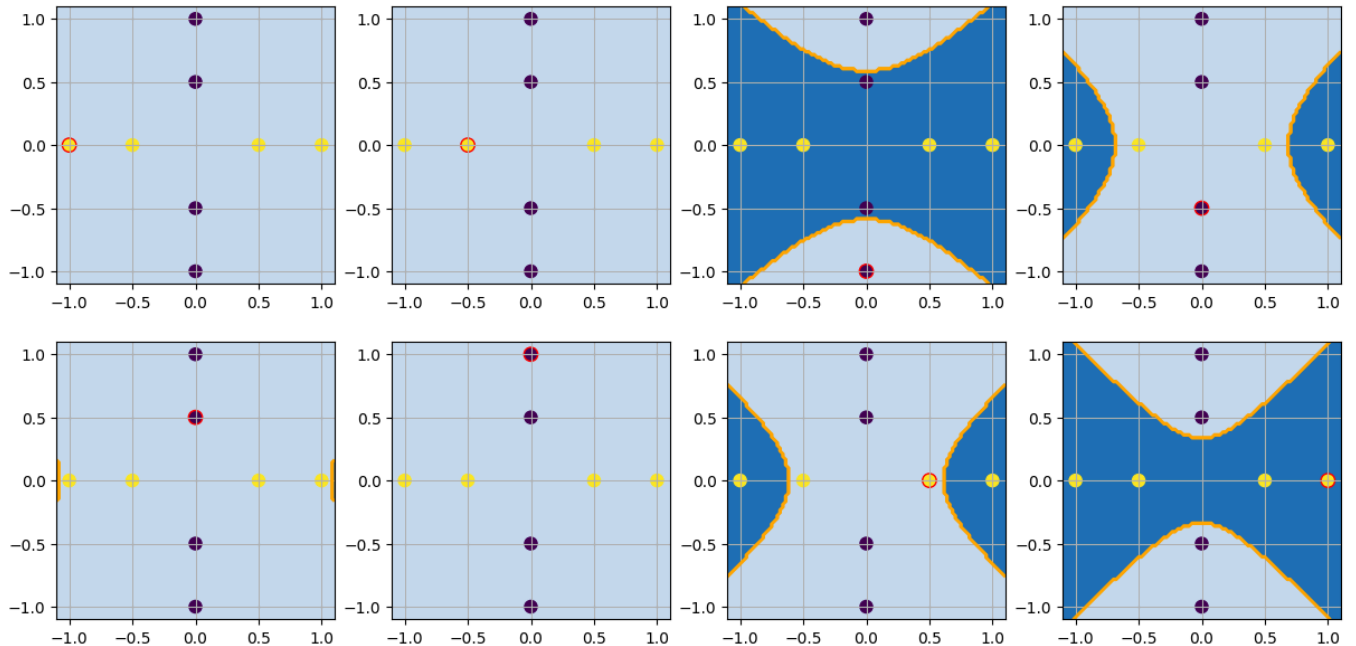
Preparación del problema y solución a máquina.

```
In [ ]: # phi = lambda x: np.array([1.0, x[0], x[1], x[0]**2, x[1]**2])
phi = lambda x: np.array([1.0, x[0]**2, x[1]**2])
Xp = np.apply_along_axis(phi, 1, X); D = Xp.shape[1]
```

```
In [ ]: N = 8; w, eta = np.zeros((9, D)), 0.75
for i in range(8):
    n = i % N
    mun = 1.0 / (1.0 + np.exp(- w[i, :] @ Xp[n, :]))
    grad = mun - y[n]
    w[i+1, :] = w[i, :] - eta * (mun - y[n]) * Xp[n, :]
    print(i+1, w[i+1])
```

```
1 [0.375 0.375 0. ]
2 [0.66368411 0.44717103 0. ]
3 [ 0.16861849  0.44717103 -0.49506561]
4 [-0.21478986  0.44717103 -0.5909177 ]
5 [-0.52255225  0.44717103 -0.6678583 ]
6 [-0.69744139  0.44717103 -0.84274744]
7 [-0.2156668  0.56761467 -0.84274744]
8 [ 0.09401581  0.87729728 -0.84274744]
```

```
In [ ]: x_min, x_max = ax.get_xlim(); y_min, y_max = ax.get_ylim()
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100), np.linspace(y_min, y_max, 100))
XX = np.c_[np.ravel(xx), np.ravel(yy)]
fig, axes = plt.subplots(2, 4, figsize=(12, 6), constrained_layout=True)
for i in range(8):
    ax = axes.flat[i]; ax.axis('off'); ax = fig.add_subplot(2, 4, i + 1)
    ax.grid(); ax.set(aspect='equal')
    z = lambda x: w[i + 1, :] @ phi(x)
    zz = np.heaviside(np.apply_along_axis(z, 1, XX), 0.0)
    ax.contour(xx, yy, zz.reshape(xx.shape), 1, colors='orange', linestyle='solid')
    cp = ax.contourf(xx, yy, zz.reshape(xx.shape), 1, cmap='Blues')
    ax.scatter(*X.T, c=y, s=64); n = i % N
    ax.scatter(X[n, 0], X[n, 1], facecolors='none', edgecolors='red', s=80)
```



5 Regresión logística

Problema. Sea un problema de clasificación en $C = 3$ clases, $y \in \{1, 2, 3\}$, y sea el modelo de regresión logística

$$p(y | \mathbf{x}; \mathbf{W}) = \text{Cat}(y | \mathcal{S}(\mathbf{W}^t \mathbf{x})) \quad \text{con} \quad \mathbf{W} = \begin{pmatrix} 1 & -1 & 0 \\ 1 & 1 & 0 \end{pmatrix}$$

Determina la probabilidad de que $\mathbf{x} = (0.5, 0.5)^t$ pertenezca a la clase 1.

Solución:

$$\begin{aligned} \mathbf{a} &= \mathbf{W}^t \mathbf{x} = \begin{pmatrix} 1 & 1 \\ -1 & 1 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \\ p(y = 1 | \mathbf{x}; \boldsymbol{\theta}) &= \mathcal{S}(\mathbf{a})_1 = \frac{e^1}{e^0 + e^0 + e^1} = \frac{e}{1 + 1 + e} = \frac{1}{1 + 2/e} = 0.5761 \end{aligned}$$

☐ Sea un problema de clasificación de objetos representados mediante vectores de $D = 2$ reales, $\mathbf{x} \in \mathbb{R}^D$, en $C = 3$ clases, $y \in \{1, 2, 3\}$. Se tiene un modelo de regresión logística sin sesgo ($\mathbf{b} = \mathbf{0}$),

$$p(y | \mathbf{x}, \boldsymbol{\theta}) = \text{Cat}(y | \boldsymbol{\mu}) \quad \text{con} \quad \boldsymbol{\mu} = \mathcal{S}(\mathbf{a}), \quad \mathbf{a} = \mathbf{W}^t \mathbf{x} \quad \text{y} \quad \mathbf{W} = \begin{pmatrix} 1 & 2 & 0 \\ 1 & 1 & 0 \end{pmatrix}$$

La probabilidad P de que $\mathbf{x} = (0.5, 0.5)^t$ pertenezca a la clase 1 es:

1. $0.00 \leq P < 0.25$
2. $0.25 \leq P < 0.50$
3. $0.50 \leq P < 0.75$
4. $0.75 \leq P \leq 1.00$

Solución: La 2.

$$\begin{aligned} \mathbf{a} &= \mathbf{W}^t \mathbf{x} = \begin{pmatrix} 1 & 1 \\ 2 & 1 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix} = \begin{pmatrix} 1 \\ 1.5 \\ 0 \end{pmatrix} \\ P &= p(y = 1 | \mathbf{x}, \boldsymbol{\theta}) = \mathcal{S}(\mathbf{a})_1 = \frac{e^1}{e^1 + e^{1.5} + e^0} = \frac{e}{8.2} = 0.3315 \end{aligned}$$

Problema. Sea $p(y | \mathbf{x}; \mathbf{W}) = \text{Cat}(y | \mathcal{S}(\mathbf{W}^t \mathbf{x}))$, $y \in \{1, 2, 3\}$, un modelo de regresión logística en notación homogénea. Sea el conjunto de datos de entrenamiento $\mathcal{D} = \{(\mathbf{x}_n, \mathbf{y}_n)\}$, donde $\mathbf{x}_n \in \mathbb{R}^D$ y $\mathbf{y}_n \in \{0, 1\}^3$ (onehot). La entropía cruzada de $\mathbf{W}$ es:

$$\mathcal{L}(\mathbf{W}) = \frac{1}{N} \sum_{n=1}^N \ell(\mathbf{y}_n, \hat{\mathbf{y}}_n) \quad \text{con} \quad \ell(\mathbf{y}_n, \hat{\mathbf{y}}_n) = - \sum_c y_{nc} \log \hat{y}_{nc}$$

Demuestra que la Jacobiana de la entropía cruzada es:

$$\frac{\partial \mathcal{L}(\mathbf{W})}{\partial \mathbf{W}} = \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n (\hat{\mathbf{y}}_n - \mathbf{y}_n)^t \quad \text{donde} \quad \hat{\mathbf{y}}_n = \boldsymbol{\mu}_n = \mathcal{S}(\mathbf{a}_n) \quad \text{con logits} \quad \mathbf{a}_n = \mathbf{W}^t \mathbf{x}_n$$

Solución. Sabemos que la Jacobiana de la entropía cruzada (con un dato) respecto de los logits es la (transpuesta de la) predicción menos la referencia. Así pues:

$$\frac{\partial \mathcal{L}(\mathbf{W})}{\partial \mathbf{W}} = \frac{1}{N} \sum_n \frac{\partial \ell(\mathbf{y}_n, \hat{\mathbf{y}}_n)}{\partial \mathbf{a}_n} \frac{\partial \mathbf{a}_n}{\partial \mathbf{W}} = \frac{1}{N} \sum_n (\hat{\mathbf{y}}_n - \mathbf{y}_n)^t \frac{\partial \mathbf{a}_n}{\partial \mathbf{W}} = \frac{1}{N} \sum_n \mathbf{x}_n (\hat{\mathbf{y}}_n - \mathbf{y}_n)^t$$

Notación matricial: con $\mathbf{Y} = (\mathbf{y}_1, \dots, \mathbf{y}_N)^t$ y aplicando la softmax por filas

$$\frac{\partial \mathcal{L}(\mathbf{W})}{\partial \mathbf{W}} = \frac{1}{N} (\mathcal{S}(\mathbf{XW}) - \mathbf{Y})^t \mathbf{X}$$

Problema. Sea $p(y | \mathbf{x}; \mathbf{W}) = \text{Cat}(y | \mathcal{S}(\mathbf{W}^t \mathbf{x}))$, $y \in \{1, 2, 3\}$, un modelo de regresión logística en notación homogénea. Sea el conjunto de datos de entrenamiento $\mathcal{D} = \{(\mathbf{x}_n, \mathbf{y}_n)\}$, donde $\mathbf{x}_n \in \mathbb{R}^D$ y $\mathbf{y}_n \in \{0, 1\}^3$ (onehot). Sabemos que la Jacobiana de la entropía cruzada es:

$$\frac{\partial \mathcal{L}(\mathbf{W})}{\partial \mathbf{W}} = \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n (\hat{\mathbf{y}}_n - \mathbf{y}_n)^t \quad \text{donde} \quad \hat{\mathbf{y}}_n = \boldsymbol{\mu}_n = \mathcal{S}(\mathbf{a}_n) \quad \text{con logits} \quad \mathbf{a}_n = \mathbf{W}^t \mathbf{x}_n$$

Supón que estamos aplicando SGD con minibatch de talla 1 para minimizar $\mathcal{L}(\mathbf{W})$. Más concretamente, nos hallamos en la iteración i con la muestra $\mathbf{x}_n = (1, 1, 1)^t$, de clase $y_n = 1$, factor de aprendizaje $\eta_i = 0.1$ y

$$\mathbf{W}_i = \begin{pmatrix} 1 & 0 & 1 \\ -1 & 1 & -1 \\ 0 & 0 & 1 \end{pmatrix}$$

Determina $\mathbf{W}_{i+1}$.

Solución:

$$\mathbf{a}_n = \mathbf{W}_i^t \mathbf{x}_n = \begin{pmatrix} 1 & -1 & 0 \\ 0 & 1 & 0 \\ 1 & -1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix} \quad \boldsymbol{\mu}_n = \mathcal{S}(\mathbf{a}_n) = \frac{1}{1 + 2e} \begin{pmatrix} 1 \\ e \\ e \end{pmatrix} = \begin{pmatrix} 0.1554 \\ 0.4223 \\ 0.4223 \end{pmatrix}$$

$$\begin{aligned} \mathbf{W}_{i+1} &= \mathbf{W}_i - \eta_i \mathbf{x}_n (\boldsymbol{\mu}_n - \mathbf{y}_n)^t = \begin{pmatrix} 1 & 0 & 1 \\ -1 & 1 & -1 \\ 0 & 0 & 1 \end{pmatrix} - 0.1 \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} (-0.8446, 0.4223, 0.4223) \\ &= \begin{pmatrix} 1 & 0 & 1 \\ -1 & 1 & -1 \\ 0 & 0 & 1 \end{pmatrix} - \begin{pmatrix} -0.0845 & 0.0422 & 0.0422 \\ -0.0845 & 0.0422 & 0.0422 \\ -0.0845 & 0.0422 & 0.0422 \end{pmatrix} = \begin{pmatrix} 1.0845 & -0.0422 & 0.9578 \\ -0.9155 & 0.9578 & -1.0422 \\ 0.0845 & -0.0422 & 0.9578 \end{pmatrix} \end{aligned}$$

6 Regresión lineal

☐ Regresión lineal predice una variable dependiente o respuesta, $y \in \mathbb{R}$, a partir de variables independientes, explicativas o regresoras, $\mathbf{x} \in \mathbb{R}^D$. Indica la respuesta incorrecta o escoge la última opción si las tres primeras son correctas:

1. Se asume que el valor esperado de y es función lineal de $\mathbf{x}$.
2. Las variables explicativas suelen incluir una constante $x_0 = 1$ para representar el modelo de forma compacta.
3. Las variables regresoras pueden ser funciones no lineales de los datos pero el modelo sigue considerándose lineal.
4. Las tres afirmaciones anteriores son correctas.

Solución: La 4.

☐ Sea $p(y | \mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y | \mu, \sigma^2)$ un modelo de regresión lineal que emplea un extractor de características para determinar la media a partir de la entrada, $\mu = \phi(\mathbf{x})$; la varianza se considera fija y conocida. Indica la respuesta incorrecta o escoge la última opción si las tres primeras son correctas:

1. El modelo es de regresión simple si la entrada es unidimensional; si no, es de regresión múltiple.
2. El modelo es de regresión polinómica de grado D si el extractor de características es una expansión polinómica de la entrada con términos de grado D a lo sumo.
3. Con pérdida L2, el estimador de Bayes óptimo es la media a posteriori y el riesgo de Bayes teórico es la varianza.
4. Las tres afirmaciones anteriores son correctas.

Solución: La 4.

Problema. Sea $p(y | x, m, c, \sigma^2) = \mathcal{N}(y | mx + c, \sigma^2)$ un modelo de regresión lineal simple de varianza fija y conocida. Ajusta la pendiente m y ordenada en origen c mediante minimización de la suma de cuadrados residual a partir de los datos de entrenamiento $\mathcal{D} = \{(x_n, y_n)\} = \{(0, 1/2), (0, 3/2), (2, 1/2), (2, 3/2)\}$.

Solución.

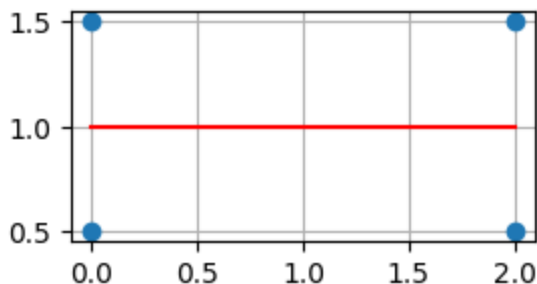
$$\begin{aligned} \mathbf{w} &= (c, m)^t & \mathbf{X} &= [1, 0; 1, 0; 1, 2; 1, 2] & \mathbf{y} &= (1/2, 3/2, 1/2, 3/2) \\ \mathbf{X}^t \mathbf{X} &= \begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 2 & 2 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 1 & 0 \\ 1 & 2 \\ 1 & 2 \end{pmatrix} = \begin{pmatrix} 4 & 4 \\ 4 & 8 \end{pmatrix} \\ (\mathbf{X}^t \mathbf{X})^{-1} &= \frac{1}{16} \begin{pmatrix} 8 & -4 \\ -4 & 4 \end{pmatrix} = \begin{pmatrix} 1/2 & -1/4 \\ -1/4 & 1/4 \end{pmatrix} \\ (\mathbf{X}^t \mathbf{X})^{-1} \mathbf{X}^t &= \begin{pmatrix} 1/2 & -1/4 \\ -1/4 & 1/4 \end{pmatrix} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 2 & 2 \end{pmatrix} = \begin{pmatrix} 1/2 & 1/2 & 0 & 0 \\ -1/4 & -1/4 & 1/4 & 1/4 \end{pmatrix} \\ \hat{\mathbf{w}} &= (\mathbf{X}^t \mathbf{X})^{-1} \mathbf{X}^t \mathbf{y} = \begin{pmatrix} 1/2 & 1/2 & 0 & 0 \\ -1/4 & -1/4 & 1/4 & 1/4 \end{pmatrix} \begin{pmatrix} 1/2 \\ 3/2 \\ 1/2 \\ 3/2 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \\ \hat{\mathbf{y}} &= \mathbf{X} \hat{\mathbf{w}} = \begin{pmatrix} 1 & 0 \\ 1 & 0 \\ 1 & 2 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix} \\ \text{RSS}(\hat{\mathbf{w}}) &= (\mathbf{y} - \hat{\mathbf{y}})^t (\mathbf{y} - \hat{\mathbf{y}}) = (-1/2, 1/2, -1/2, 1/2)(-1/2, 1/2, -1/2, 1/2)^t = 1 \end{aligned}$$

```
In [ ]: import numpy as np; np.set_printoptions(formatter={'float': lambda x: "{0:0.2f}".format(x)})
X = np.array([[1,0],[1,0],[1,2],[1,2]]); y = np.array([0.5,1.5,0.5,1.5]); eta = .05; w = np.zeros(2)
for i in range(100):
    w = w - eta * 2.0*X.T@(X@w-y)
print(w)

[1.00 0.00]
```

```
In [ ]: import numpy as np; import matplotlib.pyplot as plt
X = np.array([0.0, 0.0, 2.0, 2.0]); N = len(X); Xh = np.c_[np.ones(N), X]
y = np.array([0.5, 1.5, 0.5, 1.5]); hw = np.linalg.inv(Xh.T @ Xh) @ Xh.T @ y
y_pred = Xh @ hw; RSS = np.square(y - y_pred).sum(); print(f'hw = {hw} RSS = {RSS}')
[m, c], [r], _, _ = np.linalg.lstsq(np.c_[X, np.ones(N)], y, rcond=None)
print(f'np.linalg.lstsq: hw = {hw} RSS = {RSS}')
fig = plt.figure(figsize=(3, 1.5)); plt.plot(X, y, 'o'); plt.plot(X, m*X + c, 'r'); plt.grid(True)

hw = [1.00 0.00] RSS = 1.0
np.linalg.lstsq: hw = [1.00 0.00] RSS = 1.0
```



□ Sea un problema de regresión de objetos representados mediante vectores de $D = 1$ reales, $\mathbf{x} \in \mathbb{R}^D$. Se ha ajustado un modelo de regresión lineal con datos en notación homogénea (sesgo absorbido en $\mathbf{w}$),

$$p(y | \mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y | \mu, \sigma^2) \quad \text{con} \quad \mu = \mathbf{w}^t \mathbf{x} \quad \text{y} \quad \mathbf{x} = (1, x_1)^t \in \mathbb{R}^2$$

a partir de $N = 4$ datos, $\mathbf{X} = [1, 0; 1, 1; 1, 1; 1, 3]$ y $\mathbf{y} = (3, 1.5, 2.5, 0)^t$. Los parámetros ajustados son:

$$\hat{\mathbf{w}} = \begin{pmatrix} 3 \\ -1 \end{pmatrix} \quad \text{y} \quad \hat{\sigma}^2 = 0.125$$

El coeficiente de determinación R^2 del modelo con los datos empleados en su ajuste es:

1. $R^2 < 0.7$
2. $0.7 \leq R^2 < 0.8$
3. $0.8 \leq R^2 < 0.9$
4. $R^2 \geq 0.9$

Solución: La 4.

$$\text{RSS}(\hat{\mathbf{w}}) = N \hat{\sigma}^2 = 0.5$$

$$\bar{y} = 7/4 = 1.75$$

$$\begin{aligned} \text{TSS}(\hat{\mathbf{w}}) &= (3 - 1.75)^2 + (1.5 - 1.75)^2 + (2.5 - 1.75)^2 + (0 - 1.75)^2 \\ &= 1.5625 + 0.0625 + 0.5625 + 3.0625 = 5.25 \end{aligned}$$

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}} = 1 - 0.5/5.25 = 0.9048$$

```
In [ ]: import numpy as np; np.set_printoptions(formatter={'float': lambda x: "{0:0.2f}".format(x)})
X = np.array([[1,0],[1,1],[1,1],[1,3]]); y = np.array([3,1.5,2.5,0]); N = len(y)
w = np.linalg.inv(X.T @ X) @ X.T @ y; s = 1 / N * np.square(y - X @ w).sum(); print(w, s);
RSS = N * s; TSS = np.square(y - y.mean()).sum(); R2 = 1 - RSS / TSS; print(f'{R2:.4f}')

[3.00 -1.00] 0.125
0.9048
```

```
In [ ]: from sklearn.linear_model import LinearRegression; from sklearn.metrics import r2_score as R2
lr = LinearRegression(fit_intercept=False).fit(X, y); y_pred = lr.predict(X); print(f'{R2(y, y_pred):.4f}')

0.9048
```

□ Sea un problema de regresión de objetos representados mediante vectores de $D = 1$ reales, $\mathbf{x} \in \mathbb{R}^D$. Se ha ajustado un modelo de regresión lineal con datos en notación homogénea (sesgo absorbido en $\mathbf{w}$),

$$p(y | \mathbf{x}, \boldsymbol{\theta}) = \mathcal{N}(y | \mu, \sigma^2) \quad \text{con} \quad \mu = \mathbf{w}^t \mathbf{x} \quad \text{y} \quad \mathbf{x} = (1, x_1)^t \in \mathbb{R}^2$$

a partir de $N = 4$ datos, $\mathbf{X} = [1, 0; 1, 1; 1, 1; 1, 2]$ y $\mathbf{y} = (0, 1, 3, 4)^t$. Los parámetros ajustados son:

$$\hat{\mathbf{w}} = \begin{pmatrix} 0 \\ 2 \end{pmatrix} \quad \text{y} \quad \hat{\sigma}^2 = 0.5$$

El coeficiente de determinación R^2 del modelo con los datos empleados en su ajuste es:

1. $R^2 < 0.7$
2. $0.7 \leq R^2 < 0.8$
3. $0.8 \leq R^2 < 0.9$
4. $R^2 \geq 0.9$

Solución: La 3.

$$\text{RSS}(\hat{\mathbf{w}}) = N \hat{\sigma}^2 = 2$$

$$\bar{y} = 8/4 = 2$$

$$\text{TSS}(\hat{\mathbf{w}}) = (0 - 2)^2 + (1 - 2)^2 + (3 - 2)^2 + (4 - 2)^2 = 4 + 1 + 1 + 4 = 10$$

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}} = 1 - 2/10 = 0.8$$

```
In [ ]: import numpy as np; np.set_printoptions(formatter={'float': lambda x: "{0:0.2f}".format(x)})
X = np.array([[1,0],[1,1],[1,1],[1,2]]); y = np.array([0,1,3,4]); N = len(y)
w = np.linalg.inv(X.T @ X) @ X.T @ y; s = 1 / N * np.square(y - X @ w).sum(); print(w, s);
RSS = N * s; TSS = np.square(y - y.mean()).sum(); R2 = 1 - RSS / TSS; print(f'{R2:.4f}')

[0.00 2.00] 0.5
0.8000
```

```
In [ ]: from sklearn.linear_model import LinearRegression; from sklearn.metrics import r2_score as R2
lr = LinearRegression(fit_intercept=False).fit(X, y); y_pred = lr.predict(X); print(f'{R2(y, y_pred):.4f}')

0.8000
```